UNIDAD 10

Trabajo práctico 10 - Consultas de reservas paso a paso

1. Modalidad y punto de partida

- Individual, entrega por URL accesible de GitHub.
- Vencimiento: lunes siguiente a las 19:59.
- Copiar el TP 09 terminado como tp-10-salas-consultas-mongodb.
- Conservar fuentes, manifiesto, lockfile y herramientas; omitir .git, node_modules y .env privado al copiar.
- Usar Atlas Free, base reservas_salas_u10 y colección reservas. Conservar la configuración DNS autorizada de unidad 7.
- Pruebas en Postman de escritorio y navegador.

Preparar .env.example con marcadores y .env privado con la URI de la nueva base. No publicar credenciales. Ejecutar npm ci, npm run check y node --env-file=.env scripts/comprobar-conexion.js antes de iniciar node --env-file=.env src/index.js.

2. Objetivo

Agregar consultas al listado API sin cambiar los documentos. Resolver en este orden: filtro, orden, páginas y selección de campos. Mantener el CRUD completo, las vistas y /estado del TP 09.

No se agrega otra estructura de carpetas ni una validación genérica. Usar el patrón de las clases: consultar en el servicio y listarApi en el controlador; conservar listar para el HTML y contar para /estado.

3. Preparar una base con datos conocidos

Si está vacía, crear seis reservas por solicitudes POST separadas a http://localhost:3000/api/reservas. En Postman: Params vacío, Body > raw > JSON y Content-Type: application/json.

Enviar primero:

```
{
  "estudiante": "Ana Ejemplo",
  "email": "ana@example.com",
  "sala": "Sala Norte",
  "fecha": "2026-10-28",
  "turno": "Mañana",
  "personas": 2
}
```

Esperar 201, copiar id y revisar Location. Estado queda pendiente; no enviarlo en POST. Crear las otras cinco reservas cambiando los valores de la tabla, con la misma fecha y turno:

estudiante	email	sala	personas
Bruno Ejemplo	bruno@example.com	Sala Norte	1
Carla Ejemplo	carla@example.com	Sala Norte	2
Diego Ejemplo	diego@example.com	Sala Norte	3
Elena Ejemplo	elena@example.com	Sala Sur	1
Fabian Ejemplo	fabian@example.com	Sala Multimedia	4

Guardar los seis IDs y comprobar cada GET individual. La fecha es ilustrativa, no una validación nueva de calendario. Si los datos ya fueron cargados, no repetir los POST. No agregar semillas al arranque ni borrar la entrega anterior.

4. Paso 1 Filtrar por sala

Archivo del servicio: agregar consultar(sala) en src/servicios/reservas-mongo.js. Empezar con filtro vacío y, si llegó sala, agregar el texto recortado a filtro.sala. Buscar y representar las reservas con el mapeo público existente.

Por qué: la API necesita una condición opcional; el HTML todavía usa listar para mostrar todo.

Archivo del controlador: reemplazar el cuerpo de listarApi en src/controladores/reservas.js. Aceptar únicamente sala en esta etapa. Si llegó, exigir texto no vacío; rechazar nombres desconocidos y parámetros repetidos con 400. Si falta, consultar sin filtro.

Comprobar: sin Params, arreglo de seis; sala=Sala Norte, arreglo de cuatro; sala=Sala inexistente, arreglo vacío con 200. El filtro es igualdad exacta luego de trim, no búsqueda por fragmentos.

5. Paso 2 Agregar el orden

En los mismos dos archivos, ampliar consultar y listarApi para aceptar orden. Las únicas opciones son estudiante (predeterminada), personas-asc y personas-desc. El servicio traduce esas tres opciones a objetos de sort y añade _id ascendente para desempatar.

Por qué: el servidor elige campos conocidos y el desempate evita un orden indeterminado entre valores iguales. No enviar directamente el texto del cliente a sort.

Comprobar: Sala Norte con personas-asc devuelve 1, 2, 2 y 3. Personas-desc invierte esa secuencia; los IDs son los mismos. Orden desconocido: 400.

6. Paso 3 Agregar páginas

Ampliar solo esos dos archivos para aceptar pagina y limite. Predeterminados: pagina 1, limite 2. Convertir con Number después de exigir un único texto por parámetro; luego usar Number.isInteger y rangos pagina de 1 a 1000 y limite de 1 a 20.

Por qué: la URL transporta texto. La conversión no basta: también debemos evitar saltos negativos y tamaños fuera de lo acordado. Seguir los if explícitos de Caminatas, sin una validación genérica.

En el servicio, calcular salto=(pagina-1)*limite. Contar con countDocuments(filtro), buscar con ese mismo filtro y encadenar sort, skip, limit y lean. Devolver:

```
{
  "datos": [],
  "pagina": 1,
  "limite": 2,
  "total": 0,
  "paginas": 0
}
```

Ese ejemplo representa cero coincidencias. En general, datos contiene las reservas públicas; total cuenta todas las coincidencias y paginas es Math.ceil(total/limite). La API cambia de arreglo a este objeto; el HTML conserva su arreglo y /estado el total global.

No cambiar el contrato de GET /api/reservas/:id. Una página fuera de resultados devuelve 200 con datos vacío, no 404. No ajustar silenciosamente pagina a la última disponible.

7. Paso 4 Elegir los campos de la consulta

En consultar del servicio, agregar select con estudiante, email, sala, fecha, turno, personas y estado. Conservar _id para que la representación pública pueda construir id; seguir sin exponer _id ni __v en la respuesta.

Por qué: la consulta pide los campos necesarios para la representación pública. Esto no borra otros campos ni sustituye permisos. El cliente no puede enviar select ni elegir campos arbitrarios.

No cambiar modelos, rutas, middleware de escritura ni vistas para implementar esta selección.

8. Solicitudes en Postman, una por vez

Crear una colección TP 10 - Reservas. Guardar las solicitudes con nombres que describan el caso. En GET: Body none. En Params usar las columnas Key y Value y marcar solo las filas necesarias. Respetar espacios y acentos; comprobar la URL que genera Postman.

Key	Value
sala	Sala Norte
orden	personas-asc
pagina	1
limite	2

Enviar a http://localhost:3000/api/reservas. Con los datos iniciales: 200, dos reservas, total 4, paginas 2. Registrar sus IDs. Cambiar solo pagina a 2: otras dos reservas, mismos total y paginas. No escribir entre estas comparaciones.

Después probar por separado:

Caso	Resultado esperado
Sin Params activos	2 reservas, pagina 1, limite 2, total 6, paginas 3
Sala Norte, pagina 3, limite 2	datos vacío, total 4, paginas 2, estado 200
Sala inexistente	datos vacío, total 0, paginas 0, estado 200
pagina=0 o pagina=1.5	400
limite=21	400
Dos filas pagina activas	400
sala vacía o repetida	400
orden=otro o select=email	400

El listado exitoso expone únicamente datos, pagina, limite, total y paginas; cada reserva conserva id y los siete campos públicos. Con las cantidades ya verificadas, comprobar POST, PUT, PATCH y DELETE heredados sobre datos de ejercicio. Si una escritura cambia la cantidad, registrar el nuevo total; no seguir esperando seis.

9. Continuidad y pequeña decisión de diseño

El formulario conserva redirección 302 al crear y 400 con valores conservados al fallar. HTML, CSS, layout, detalle, GET individual y /estado siguen funcionando. Los rechazos de consulta no escriben datos. Reiniciar y comprobar el mismo ID sin duplicar cargas.

En el README, agregar un párrafo breve: si incorporáramos información de las salas, ¿qué datos describirían una sala y cuáles describen una reserva? ¿Por qué tendría sentido considerar una referencia o datos embebidos? Se evalúa la justificación inicial; no se implementa otro modelo ni populate.

No se requieren autenticación, nuevos paquetes, búsqueda aproximada, operadores libres, orden dinámico, metadatos adicionales ni otra interfaz de paginación en HTML.

10. Evidencia y evaluación

Entregar código fuente y README con instalación, variables sin secretos, contrato final y solicitudes reproducibles de Postman. Adjuntar capturas o exportación de la colección con respuestas documentadas. No incluir URI privada, node_modules ni datos personales reales.

Criterio	Peso
Filtro por sala y orden limitado, con desempate	25%
Páginas, validación simple y metadatos coherentes	30%
Proyección y conservación de CRUD, HTML y /estado	20%
Datos de ejercicio, pruebas y documentación reproducible	20%
Justificación conceptual de relaciones	5%

Antes de entregar: npm run check sin errores, repositorio accesible y resultados contrastados con los datos que efectivamente quedaron guardados.

Material de apoyo

- Clase 1: filtro y orden.
- Clase 2: páginas y campos.
- Clínica del viernes.